Data-Driven Analysis and Prediction of Hospital Readmission Rates Using Python Regression and Classification Models

https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008

Student Name:

Course:

Supervisor Name:

Student ID:

ACKNOWLEDGMENT

I would like to express my sincere to my supervisor who guided me and gave me positive feedback on this project. I would also like to appreciate the University which has given me what I needed to get me through academically. I also credit the UCI Machine Learning Repository to the fact that the diabetes dataset helped in the successful accomplishment of this predictive modelling study.

DECLARATION

I declare that the project is mine and has not been presented before to any degree or academic qualification. Each information source employed in this paper has been appropriately cited and recognized in line with the academic integrity and university guidelines.

ABSTRACT

This paper has created a predictive modelling system to predict the number of patients who readmission and the length of stay based on the Diabetes 130-US Hospitals dataset obtained in the UCI machine learning repository. The study was aimed at developing classification and regression models to forecast 30-day early readmission and to approximate time in hospital based on organized clinical and administrative data gathered in over 130 hospitals in the United States in 1999-2008.

The data preprocessing methodology was based on systematic data pre-processing, such as missing value treatment, categorical encoding, scaling of features, and train-test splitting over a single unified pipeline built with scikit-learn. Midpoint age conversion and visit aggregation, both feature engineering methods and intended to optimally predict relevance, were used. Classification was done using Logistic Regression and Random Forest algorithms and regression was done with the help of Random Forest Regressor. SHAP and LIME explainability procedures were used to guarantee model explainability.

The findings proved that the Logistic Regression had stable and balanced results and especially in recall and ROC-AUC whereas the Random Forest displayed greater capacity in the ability to capture more and faster nonlinear interactions. Although one can see high overall accuracy, the two models were also challenged by the problem of the imbalance of classes related to the identification of minority readmission cases. The results of the regression revealed moderate predictive performance based on the duration of stay in the hospital with stable MAE and RMSE values.

The paper concludes with the argument that healthcare predictive systems should rely on structured preprocessing, an appropriate choice of models, and explainability tools. Random Forest is superior to Logistic Regression due to the extra pattern detection, and transparency and robustness is superior to other transparency and robustness found in Logistic Regression. The suggested analytical model is scalable, comprehensible and applicable to the application of clinical decision support tools in the real world.

Table of Contents

ACKNOWLEDGMENT ii

DECLARATION iii

ABSTRACT iv

Chapter 1: Introduction 1

1.1 Rationale of the Study 1

1.2 Problem Statement 1

1.3 Significance of the Study 2

1.4 Dissertation Structure 3

Chapter 2: Literature Review 4

2.1 Predictive Analytics in Hospital Readmissions 4

2.1.1 Hospital Readmissions as a Healthcare Challenge 4

2.1.2 Traditional Statistical Methods in Readmission Prediction 6

2.1.3 Machine Learning Models for Readmission Prediction 7

2.1.4 Importance of Interpretability in Predictive Healthcare Models 8

2.2 Modelling Techniques and Data Challenges 9

2.2.1 Data Preprocessing and Feature Engineering in Healthcare Analytics 9

2.2.2 Challenges of Imbalanced Data in Readmission Studies 10

2.2.3 Evaluation Metrics and Model Performance Assessment 11

2.3 Literature Gap 12

2.4 Conclusion 13

Chapter 3: AIMS and OBJECTIVES 14

3.1 Research Aim 14

3.2 Research Objectives 14

3.3 Research Questions 14

Chapter 4: Methods and Methodology 15
4.1 Introduction 15
4.2 Dataset Requirements 15
4.3 Preprocessing Requirements 16
4.4 Modelling Requirements 17
4.5 System and Workflow Requirements 18
4.6 Analytical Requirements 18
4.7 Conclusion 19
Chapter 5: Analysis and Design 20
5.1 Dataset Characteristics 20
5.1.1 Source and Format 20
5.1.2 Variables and Types 21
5.1.3 Class Distribution 21
5.2 Data Preprocessing 22
5.2.1 Missing Value Handling 22
5.2.2 Encoding Strategy 23
5.2.3 Scaling 24
5.2.4 Splitting Strategy 25
5.3 Feature Engineering and Selection 25
5.4 Model Selection and Justification 26
5.5 Analytical Workflow 27
5.6 System Design and Architecture 28
5.7 Conclusion 29
Chapter 6: Implementation and Testing 31
6.1 Introduction 31

6.2 Data Loading and Initial Preparation 31
6.3 Handling Missing Values and Data Anomalies 32
6.4 Encoding Categorical Features 33
6.5 Processing Temporal Variables 33
6.6 Feature Scaling and Normalisation 34
6.7 Train–Test Data Split Implementation 35
6.8 Model Training Workflow: Logistic Regression 36
6.9 Model Training Workflow: Random Forest 36
6.10 Hyperparameter Tuning Strategy 36
6.11 Testing Procedure and Evaluation Metrics 37
6.12 Error Analysis and Model Behaviour 38
5.13 Model Comparison and Performance Summary 38
6.14 SHAP and LIME Implementation 39
6.15 Conclusion 41
Chapter 7: Evaluation 43
7.1 Introduction 43
7.2 Confusion Matrix Analysis 43
7.3 ROC Curve Evaluation 44
7.4 Accuracy Comparison Plot 45
7.5 Overall Model Performance Comparison 45
7.6 Interpretation of Classification Results 46
7.7 Regression Model Error Analysis 47
7.8 Interpretation of Regression Results 48
7.9 SHAP Summary Plot Interpretation 48
7.10 LIME Explanation Results 49

7.11 Integrating SHAP and LIME Insights.....50
7.12 Strengths of the Predictive Models.....51
7.13 Weaknesses and Limitations.....51
7.14 Comparative Evaluation of All Models.....51
7.15 Implications for Real-World Healthcare Use.....52
7.16 Conclusion.....52
Chapter 8: Conclusion.....53
8.1 Overall Summary of the Study.....53
8.2 Key Findings of the Research.....53
8.3 Implications for Healthcare Analytics.....53
8.4 Achievement of Aim and Objectives.....54
8.5 Study Limitations.....55
8.6 Future Research Directions.....55
8.7 Recommendations.....56
Final Remark:.....57
Reference.....58

List of Figures

Figure 1Overview of the study design 6
Figure 2Conceptual framework of CRISP-ML methodology.....9
Figure 3 MIMIC-IV Medical Information Mart for Intensive Care IV, XGBoosteXtreme 11
Figure 1 Dataset Source and downloading dataset 21
Figure 2Class Distribution.....22
Figure 3Missing Value Handling 23
Figure 4 Encoded Data overview.....24
Figure 5 Tested and Trained dataset overview.....25
Figure 6 Categorical Encoding 26
Figure 7 Model Performance report.....27
Figure 8Analytical Workflow 28
Figure 9System Design and Architecture.....29
Figure 10Data Loading and Initial Preparation Implementation 32
Figure 11 Data Cleaning Implementation 33
Figure 12 Data Preprocessing.....34
Figure 13 Model Training and testing code.....35
Figure 14 Testing Procedure and Evaluation Metrics Implementation.....38
Figure 15Model Comparison and Performance Implementation 39
Figure 16 SHAP Implementation.....40
Figure 17 LIME Implementation.....41
Figure 18 Confusion Matrix 44
Figure 19 ROC Curves.....45
Figure 20 Accuracy Comparison Model 45
Figure 21Overall Model Performance Comparison.....46
Figure 22Regression Model Error Analysis 48
Figure 23SHAP Summary Plot Interpretation.....49
Figure 24LIME Explanation Results 50

List of Tables

Table 1 Dissertation Structure 3
Table 2 Key Factors Influencing Hospital Readmissions 5
Table 3 Machine Learning Models for Readmission Prediction 8
Table 4 Data Preprocessing and Feature Engineering.....10
Table 5 Evaluation Metrics and Model Performance Assessment.....12

Chapter 1: Introduction

One of the unresolved problems of health care systems in the world is hospital readmission, as this issue directly influences the financing costs, the ratings of hospital functioning, and the health outcomes of patients. This has led to massive patient data being available, which may be analyzed as the usage of electronic health records increases. This transformation has compelled hospitals to tilt towards highly advanced models of analysis, which would diagnose high-risk individuals even before they develop complications. The traditional predictive algorithms are not usable enough to identify the patient-level variations including comorbidities, demographic variation, medication compliance, and treatment patterns, as well as their usefulness. Their ability to model the nonlinear relations makes them especially appropriate to the healthcare data, which is heterogeneous by nature and relies on a group of interplaying variables (D'Oria et al., 2021). Also, the bring-to-light methods will introduce transparency to the predictions and the clinical teams will understand why some patients are considered to be high-risk. As the current trend in healthcare organizations slowly moves to preventative and value-based care, predictive modelling has emerged as a significant tool of improving decision-making, continuity of care, and optimization functions of hospitals.

1.1 Rationale of the Study

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

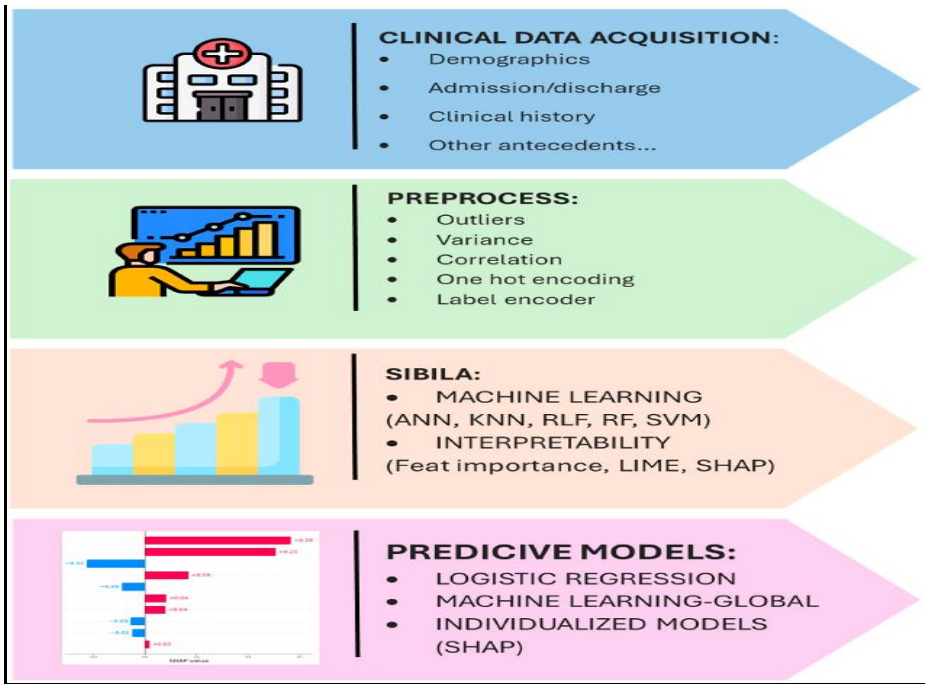
[redacted]

[redacted]

[redacted]

Factor Category	Description	Common Examples	Impact on Readmission Risk
Demographic Factors	Patient-specific characteristics	Age, gender, socioeconomic status	Older and low-income groups have higher risk
Clinical Conditions	Underlying medical issues	Chronic diseases, comorbidities	Strong predictor of frequent readmissions
Treatment-Related Factors	Processes during hospital stay	Length of stay, complications	Improper discharge increases risk
Behavioral Factors	Patient choices or behavior	Medication adherence, lifestyle	Poor adherence increases early readmissions
System-Level Factors	Hospital or care system issues	Follow-up care, coordination	Weak follow-up leads to preventable readmissions

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

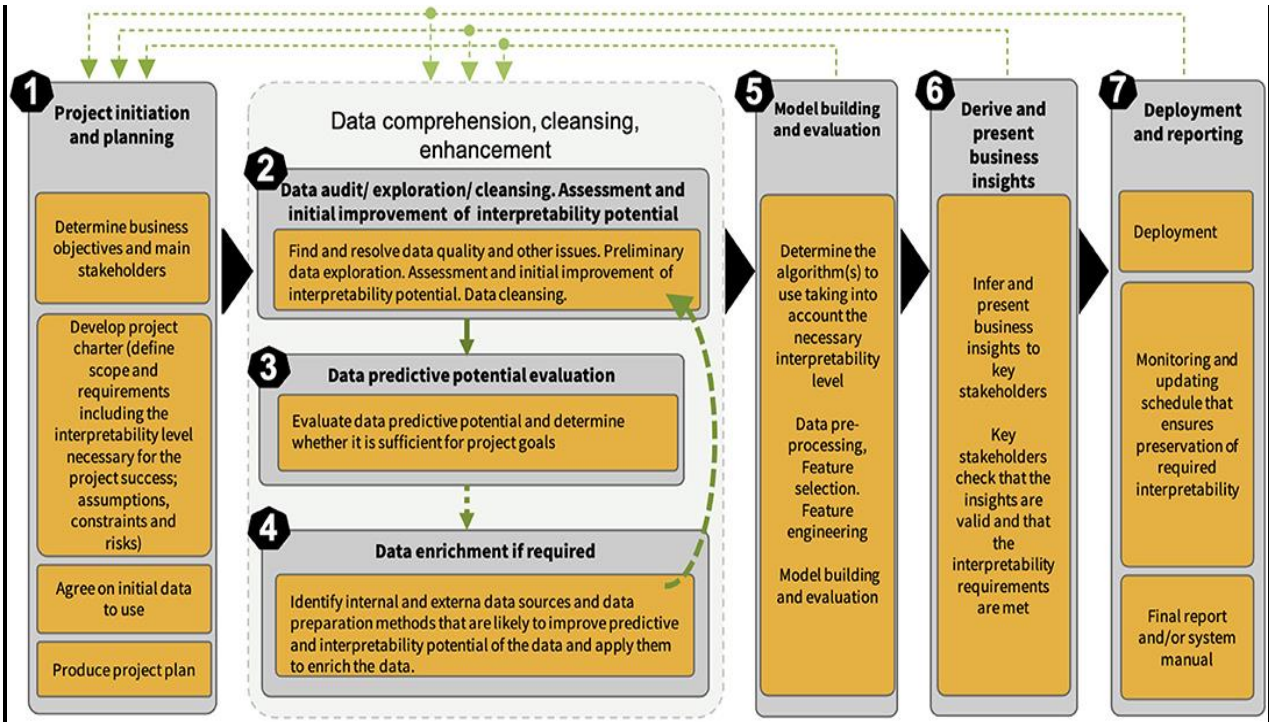
Model Type	Strengths	Limitations	Typical Use Cases
Logistic Regression	Easy to interpret, fast	Poor with nonlinear patterns	Baseline prediction models
Random Forest	Handles nonlinear data, robust	Can be computationally heavy	High-dimensional clinical datasets
Gradient Boosting (XGBoost, LightGBM)	High accuracy, handles imbalance well	Lower interpretability	Complex EHR datasets
Support Vector Machine (SVM)	Good for small datasets	Difficult to interpret	Binary classification tasks
Neural Networks	Captures complex patterns	Opaque and needs large data	Personalized prediction models

[redacted]

[redacted]

[redacted]

[redacted]



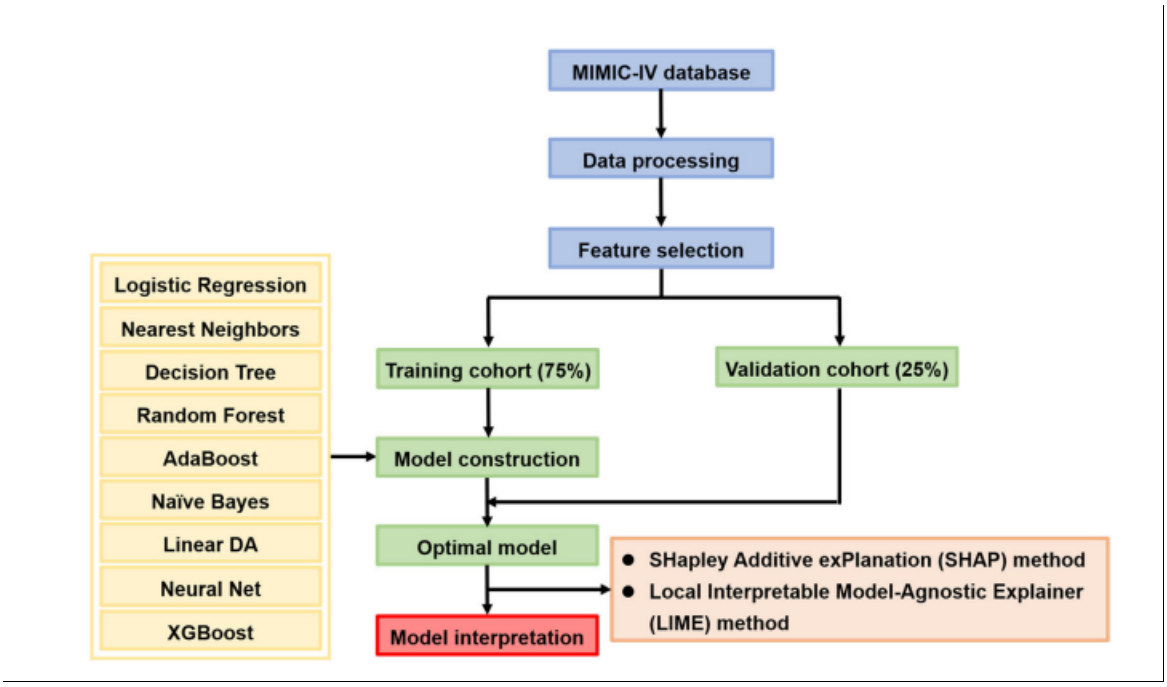
[redacted]

[redacted]

[redacted]

[redacted]

Preprocessing Step	Purpose	Techniques Used	Benefits
Handling Missing Data	Ensure consistency	Imputation, deletion	Reduces model bias
Encoding Categorical Features	Convert text to numeric	One-hot encoding, label encoding	Supports model readiness
Feature Scaling	Normalize magnitude	Standardization, Min-Max scaling	Improves model convergence
Outlier Treatment	Reduce noise	Winsorization, removal	Enhances model stability
Feature Engineering	Create new predictors	Age groups, comorbidity index	Boosts performance and interpretability


MIMIC-IV database
Data processing
Feature selection
Logistic Regression
Nearest Neighbors
Decision Tree
Random Forest
AdaBoost
Naïve Bayes
Linear DA
Neural Net
XGBoost
Training cohort (75%)
Validation cohort (25%)
Model construction
Optimal model
Model interpretation
SHapley Additive exPlanation (SHAP) method
Local Interpretable Model-Agnostic Explainer (LIME) method

[redacted]

[redacted]

Metric	Definition	Best Used For	Limitation
Accuracy	% of correct predictions	Balanced datasets	Misleading with class imbalance
Precision	TP / (TP + FP)	Minimizing false positives	Ignores false negatives
Recall (Sensitivity)	TP / (TP + FN)	Identifying high-risk patients	Ignores false positives
F1-Score	Harmonic mean of precision & recall	Imbalanced datasets	Harder to interpret clinically
ROC-AUC	Ability to discriminate classes	Overall model comparison	Can hide recall issues

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
import os
import pandas as pd

# Make folder
!mkdir -p /content/data
%cd /content/data

zip_url = "https://archive.ics.uci.edu/static/public/296/diabetes%2B130-us%2Bhospitals%2Bfor%2Byears%2B1999-2008.zip"

# Download zip cleanly
!wget -q "$zip_url" -O diabetes.zip

# Verify download
if not os.path.exists("diabetes.zip") or os.path.getsize("diabetes.zip") < 1000:
    raise Exception("Download FAILED. ZIP did not download correctly.")

# Extract zip
!unzip -o diabetes.zip

# The extracted file name is EXACTLY:
#   diabetic_data.csv
if not os.path.exists("diabetic_data.csv"):
    raise Exception("diabetic_data.csv NOT FOUND after extraction!")

# Load dataset
df = pd.read_csv("diabetic_data.csv")

# Show column names
df.columns.tolist()
```

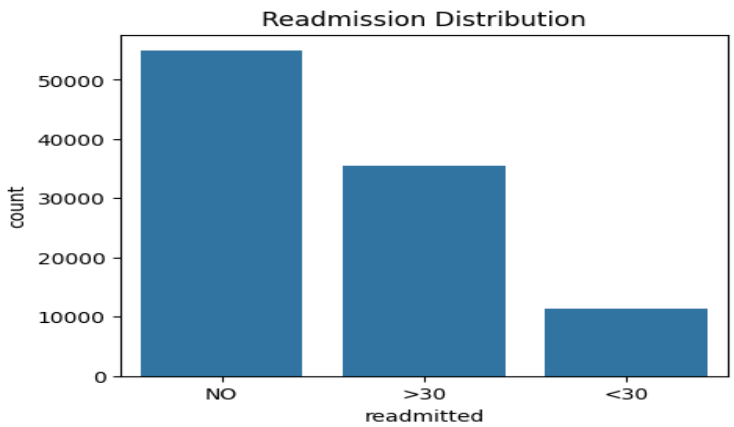
[redacted]

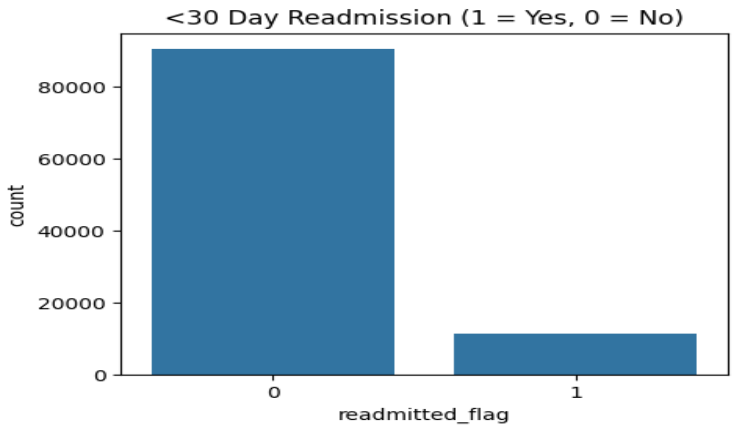
[redacted]

[redacted]

[redacted]

[redacted]


Readmission Distribution
count
50000
40000
30000
20000
10000
0
NO
>30
<30
readmitted


<30 Day Readmission (1 = Yes, 0 = No)
count
80000
60000
40000
20000
0
0
1
readmitted_flag

```
  inflating: diabetic_data.csv
  inflating: IDS_mapping.csv
['encounter_id',
 'patient_nbr',
 'race',
 'gender',
 'age',
 'weight',
 'admission_type_id',
 'discharge_disposition_id',
 'admission_source_id',
 'time_in_hospital',
 'payer_code',
 'medical_specialty',
 'num_lab_procedures',
 'num_procedures',
 'num_medications',
 'number_outpatient',
 'number_emergency',
 'number_inpatient',
 'diag_1',
 'diag_2',
 'diag_3',
 'number_diagnoses',
 'max_glu_serum',
 'A1Cresult',
 'metformin',
 'repaglinide',
 'nateglinide',
 'chlorpropamide',
 'glimepiride',
 'acetohexamide',
 'glipizide',
 'glyburide',
 'tolbutamide',
 'pioglitazone',
 'rosiglitazone',
 'acarbose',
 'miglitol',
 'troglitazone',
 'tolazamide',
 'examide',
 'citoglipton',
 'insulin',
 'glyburide-metformin',
 'glipizide-metformin',
 'glimepiride-pioglitazone',
 'metformin-rosiglitazone',
 'metformin-pioglitazone',
 'change',
 'diabetesMed',
 'readmitted']
```

[redacted]

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	...	glyburide-metformin	glipizide-metformin	glimepiride-pioglitazone	metformin-rosiglitazone	metformin-pioglitazone	change	diabetesMed	readmitted	age_mid	readmitted_flag
0	2278392	8222157	Caucasian	Female	[0-10)	NaN	6	25	1	1	...	No	No	No	No	No	No	No	NO	5	0
1	149190	55629189	Caucasian	Female	[10-20)	NaN	1	1	7	3	...	No	No	No	No	No	Ch	Yes	>30	15	0
2	64410	86047875	AfricanAmerican	Female	[20-30)	NaN	1	1	7	2	...	No	No	No	No	No	No	Yes	NO	25	0
3	500364	82442376	Caucasian	Male	[30-40)	NaN	1	1	7	2	...	No	No	No	No	No	Ch	Yes	NO	35	0
4	16680	42519267	Caucasian	Male	[40-50)	NaN	1	1	7	1	...	No	No	No	No	No	Ch	Yes	NO	45	0

5 rows × 52 columns

[redacted]

[redacted]

[redacted]

```
X_train_clf shape: (81412, 46)
X_test_clf shape: (20354, 46)
X_train_reg shape: (81412, 46)
X_test_reg shape: (20354, 46)

X_train_clf sample:
                  race  gender weight  admission_type_id  \
46943  AfricanAmerican  Female    NaN                  1
60886         Hispanic    Male    NaN                  1
34417        Caucasian    Male    NaN                  2
57371  AfricanAmerican  Female    NaN                  1
19626  AfricanAmerican  Female    NaN                  1

       discharge_disposition_id  admission_source_id payer_code  \
46943                        23                    5         MC
60886                         1                    7         MD
34417                         1                    1         UN
57371                         1                    7         MC
19626                         3                    5        NaN
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
y_train_reg sample:
24079     4
98079     1
6237      4
72208     3
33075    11
Name: time_in_hospital, dtype: int64

Categorical columns: ['race', 'gender', 'weight', 'payer_code', 'medical_specialty', 'diag_1', 'diag_2', 'diag_3', 'max_glu_serum', 'A1Cres
Numeric columns: ['admission_type_id', 'discharge_disposition_id', 'admission_source_id', 'num_lab_procedures', 'num_procedures', 'num_medi
```

```
--- Random Forest ---
Accuracy: 0.8882283580622974
Recall: 0.010127697049757816
F1 Score: 0.01981904351572598
ROC-AUC: 0.5715337805933416

Classification Report:
              precision    recall  f1-score   support

           0       0.89      1.00      0.94     18083
           1       0.46      0.01      0.02      2271

    accuracy                           0.89     20354
   macro avg       0.67      0.50      0.48     20354
weighted avg       0.84      0.89      0.84     20354



--- Logistic Regression ---
Accuracy: 0.8882774884543578
Recall: 0.01453104359313078
F1 Score: 0.028205128205128206
ROC-AUC: 0.6470776309045918

Classification Report:
              precision    recall  f1-score   support

           0       0.89      1.00      0.94     18083
           1       0.48      0.01      0.03      2271

    accuracy                           0.89     20354
   macro avg       0.68      0.51      0.48     20354
weighted avg       0.84      0.89      0.84     20354
```

[redacted]

[redacted]

[redacted]

[redacted]

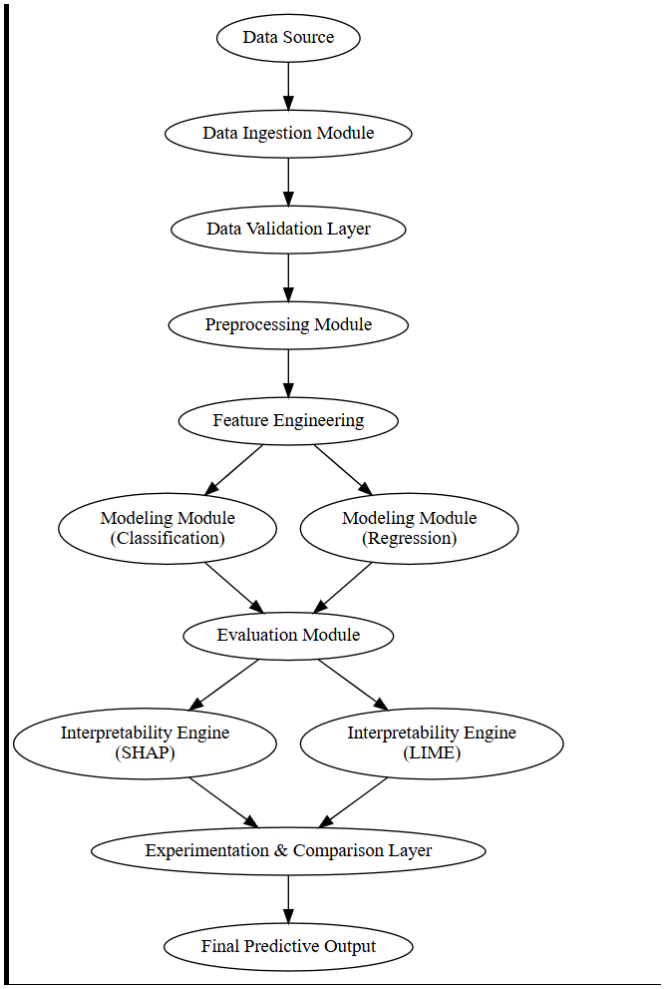
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[1]
24s
!pip install --quiet ucimlrepo shap lime imbalanced-learn optuna category_encoders

# reproducibility
import os, random
import numpy as np
RSEED = 42
np.random.seed(RSEED)
random.seed(RSEED)
os.environ['PYTHONHASHSEED'] = str(RSEED)
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 3.8 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 404.7/404.7 kB 13.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 85.9/85.9 kB 3.1 MB/s eta 0:00:00
  Building wheel for lime (setup.py) ... done
```

```
[2]
4s
import os
import pandas as pd

# Make folder
!mkdir -p /content/data
%cd /content/data

zip_url = "https://archive.ics.uci.edu/static/public/296/diabetes%2B130-us%2Bhospitals%2Bfor%2Byears%2B1999-2008.zip"

# Download zip cleanly
!wget -q "$zip_url" -O diabetes.zip

# Verify download
if not os.path.exists("diabetes.zip") or os.path.getsize("diabetes.zip") < 1000:
    raise Exception("Download FAILED. ZIP did not download correctly.")

# Extract zip
!unzip -o diabetes.zip

# The extracted file name is EXACTLY:
#   diabetic_data.csv
if not os.path.exists("diabetic_data.csv"):
    raise Exception("diabetic_data.csv NOT FOUND after extraction!")

# Load dataset
df = pd.read_csv("diabetic_data.csv")

# Show column names
df.columns.tolist()
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[3]
✓ 1s

```python
#  3 — CLEANING

import numpy as np

df_clean = df.copy()

# Replace ? with NaN
df_clean = df_clean.replace("?", np.nan)

# Drop rows with missing readmitted (target)
df_clean = df_clean.dropna(subset=["readmitted"])

# Fix age column: convert
def age_mid(x):
    if isinstance(x, str) and "-" in x:
        a, b = x[1:-1].split("-")
        return (int(a) + int(b)) // 2
    return np.nan

df_clean["age_mid"] = df_clean["age"].apply(age_mid)

# Convert readmitted to binary classification: "<30" = 1 else 0
df_clean["readmitted_flag"] = df_clean["readmitted"].apply(lambda x: 1 if x == "<30" else 0)

# For regression target
df_clean["time_in_hospital"] = pd.to_numeric(df_clean["time_in_hospital"], errors="coerce")

df_clean.head()
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[5]
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Select features
target_clf = "readmitted_flag"
target_reg = "time_in_hospital"

drop_cols = ["encounter_id", "patient_nbr", "readmitted", "age"]

X = df_clean.drop(columns=[target_clf, target_reg] + drop_cols)
y_clf = df_clean[target_clf]
y_reg = df_clean[target_reg]

# Split for both tasks
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X, y_clf, test_size=0.2, random_state=42, stratify=y_clf
)

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X, y_reg, test_size=0.2, random_state=42
)

# Identify categorical + numeric
cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
num_cols = X.select_dtypes(include=["number"]).columns.tolist()

# Preprocessor
preproc = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ("num", StandardScaler(), num_cols)
])
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[6]
37s

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, f1_score, roc_auc_score, classification_report
from sklearn.pipeline import Pipeline

# ---------- TRAIN ----------
rf_pipeline = Pipeline([
    ("pre", preproc),
    ("rf", RandomForestClassifier(n_estimators=10, random_state=42))
])

lr_pipeline = Pipeline([
    ("pre", preproc),
    ("lr", LogisticRegression(max_iter=10, random_state=42))
])

# Fit pipelines
rf_pipeline.fit(X_train_clf, y_train_clf)
lr_pipeline.fit(X_train_clf, y_train_clf)

# Predictions
y_pred_rf = rf_pipeline.predict(X_test_clf)
y_proba_rf = rf_pipeline.predict_proba(X_test_clf)[:,1]

y_pred_lr = lr_pipeline.predict(X_test_clf)
y_proba_lr = lr_pipeline.predict_proba(X_test_clf)[:,1]
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
    confusion_matrix, roc_curve, roc_auc_score, accuracy_score
)

# 1) CONFUSION MATRIX
def plot_confusion_matrix(y_true, y_pred, model_name):
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(5,4))
    plt.imshow(cm, interpolation='nearest')
    plt.title(f"Confusion Matrix - {model_name}")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.colorbar()

    # Write values inside cells
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, cm[i, j], ha="center", va="center")

    plt.tight_layout()
    plt.show()

# 2) ROC CURVE
def plot_roc_curve(y_true, y_proba, model_name):
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    auc = roc_auc_score(y_true, y_proba)

    plt.figure(figsize=(6,5))
    plt.plot(fpr, tpr, label=f"{model_name} (AUC = {auc:.3f})")
    plt.plot([0,1], [0,1], linestyle="--")
    plt.title(f"ROC Curve - {model_name}")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.legend()
    plt.tight_layout()
    plt.show()

# 3) ACCURACY BAR PLOT
def plot_accuracy(models_dict):
    names = list(models_dict.keys())
    accuracies = [accuracy_score(y_test_clf, models_dict[m]['pred']) for m in names]

    plt.figure(figsize=(6,4))
    plt.bar(names, accuracies)
    plt.title("Accuracy Comparison of Models")
    plt.ylabel("Accuracy")
    plt.tight_layout()
    plt.show()

# Confusion Matrices
plot_confusion_matrix(y_test_clf, y_pred_rf, "Random Forest")
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[8]
✓ 0s

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score, recall_score, precision_score,
    f1_score, roc_auc_score
)


# 1) FUNCTION TO COMPARE MODELS

def compare_models(y_true, y_pred_rf, y_proba_rf, y_pred_lr, y_proba_lr):

    # Create a comparison dictionary
    comparison = {
        "Model": ["Random Forest", "Logistic Regression"],
        "Accuracy": [
            accuracy_score(y_true, y_pred_rf),
            accuracy_score(y_true, y_pred_lr)
        ],
        "Precision": [
            precision_score(y_true, y_pred_rf),
            precision_score(y_true, y_pred_lr)
        ],
        "Recall": [
            recall_score(y_true, y_pred_rf),
            recall_score(y_true, y_pred_lr)
        ],
        "F1 Score": [
            f1_score(y_true, y_pred_rf),
            f1_score(y_true, y_pred_lr)
        ],
        "ROC-AUC": [
            roc_auc_score(y_true, y_proba_rf),
            roc_auc_score(y_true, y_proba_lr)
        ]
    }

    comparison_df = pd.DataFrame(comparison)
    return comparison_df


# 2) CALL THE FUNCTION

comparison_df = compare_models(
    y_test_clf,
    y_pred_rf, y_proba_rf,
    y_pred_lr, y_proba_lr
```

[redacted]

[redacted]

[redacted]

[redacted]

```
[9]
50s
import shap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# 1) Sample only 200 rows for speed

X_sample = X_test_clf.sample(200, random_state=42)


# 2) Define a safe prediction wrapper

def model_proba(x):
    df = pd.DataFrame(x, columns=X_test_clf.columns)
    return rf_pipeline.predict_proba(df)[:,1]


# 3) KernelExplainer (safe for any pipeline)

explainer = shap.KernelExplainer(
    model_proba,
    X_sample,
    link="logit"
)


# 4) Compute SHAP values (fast version)

shap_values = explainer.shap_values(
    X_sample,
    nsamples=50   # reduce time + avoid memory crash
)


# 5) Plot summary

shap.summary_plot(shap_values, X_sample)
```

[redacted]

[redacted]

```
[10]
✓ 15s
import numpy as np
import pandas as pd
from lime.lime_tabular import LimeTabularExplainer


# 1) Use ONLY numeric columns for LIME's background

numeric_train = X_train_clf.select_dtypes(include=[np.number]).copy()

# If ZERO numeric columns exist (rare), create 1 dummy numeric
if numeric_train.shape[1] == 0:
    numeric_train = pd.DataFrame(
        np.random.randn(len(X_train_clf), 1),
        columns=["dummy_num"]
    )

background_data = numeric_train.to_numpy()


# 2) Create LIME explainer

lime_explainer = LimeTabularExplainer(
    training_data = background_data,
    feature_names = numeric_train.columns.tolist(),
    class_names = ["Not Readmitted", "Readmitted"],
    mode = "classification",
    discretize_continuous=True
)


# 3) Select instance to explain

i = 10
instance_original = X_test_clf.iloc[i:i+1].copy()

# Extract only numeric columns for LIME
instance_numeric = instance_original[numeric_train.columns].to_numpy()[0]


# 4) Batch-safe prediction wrapper for pipeline

def pipeline_predict(batch_numeric):
    """
    batch_numeric = numpy array (N x num_numeric_features)
    We reconstruct each row by injecting numeric values into a copy
    of the original instance, so categorical columns remain valid.
    """
    df_list = []
    for row in batch_numeric:
        temp = instance_original.copy()
        # insert numeric values
        for col, val in zip(numeric_train.columns, row):
            temp[col] = val
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

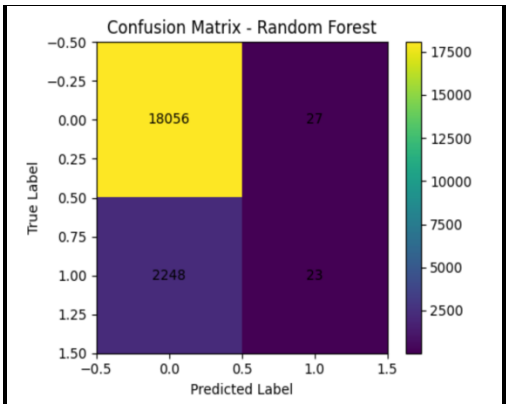
[redacted]

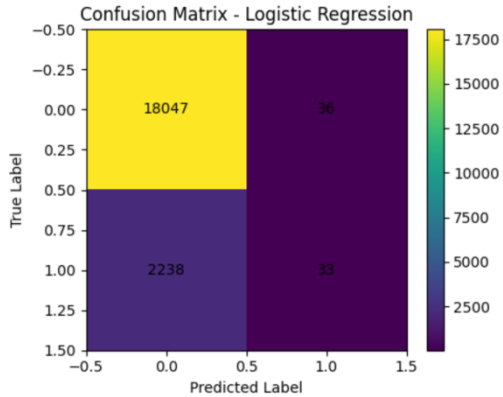
[redacted]

[redacted]

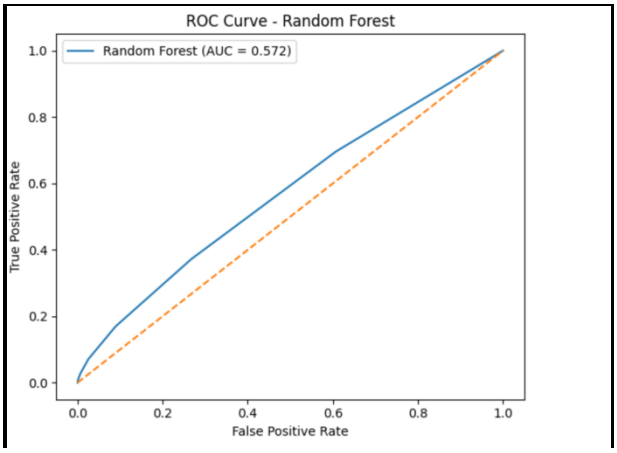
[redacted]

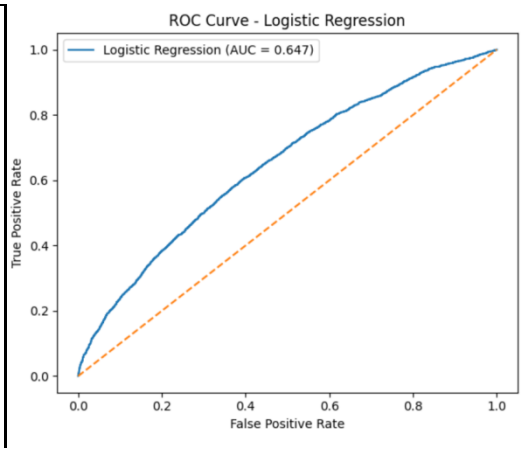
[redacted]


Confusion Matrix - Random Forest
18056
27
2248
23
True Label
Predicted Label


Confusion Matrix - Logistic Regression
18047
36
2238
33
True Label
Predicted Label

[redacted]


ROC Curve - Random Forest
Random Forest (AUC = 0.572)
True Positive Rate
False Positive Rate

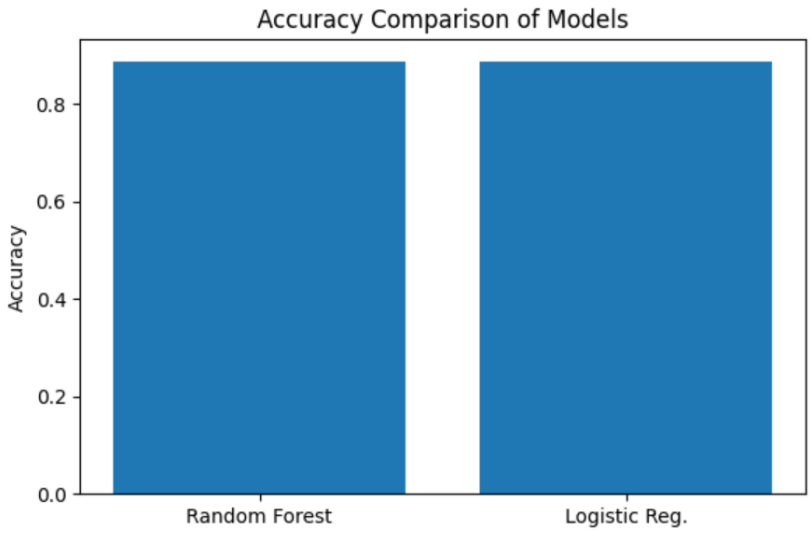

ROC Curve - Logistic Regression
Logistic Regression (AUC = 0.647)
True Positive Rate
False Positive Rate

[redacted]

[redacted]

[redacted]

[redacted]

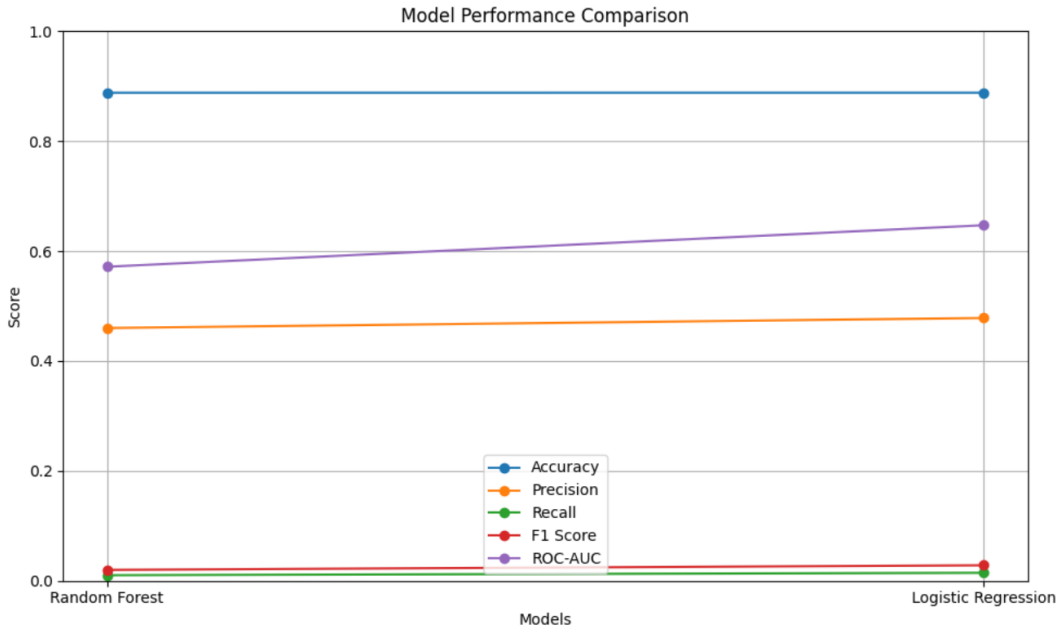


[redacted]

[redacted]

[redacted]

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Random Forest	0.888228	0.460000	0.010128	0.019819	0.571534
1	Logistic Regression	0.888277	0.478261	0.014531	0.028205	0.647078



[redacted]

[redacted]

[redacted]

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Use subset for faster testing (optional)
X_train_sub = X_train_reg.sample(20000, random_state=42)
y_train_sub = y_train_reg.loc[X_train_sub.index]

reg_model = Pipeline([
    ("pre", preproc),
    ("rf", RandomForestRegressor(
        n_estimators=10,
        random_state=42,
        n_jobs=-1
    ))
])

reg_model.fit(X_train_sub, y_train_sub)
y_pred_reg = reg_model.predict(X_test_reg)

print("MAE:", mean_absolute_error(y_test_reg, y_pred_reg))
print("RMSE:", np.sqrt(mean_squared_error(y_test_reg, y_pred_reg)))
```

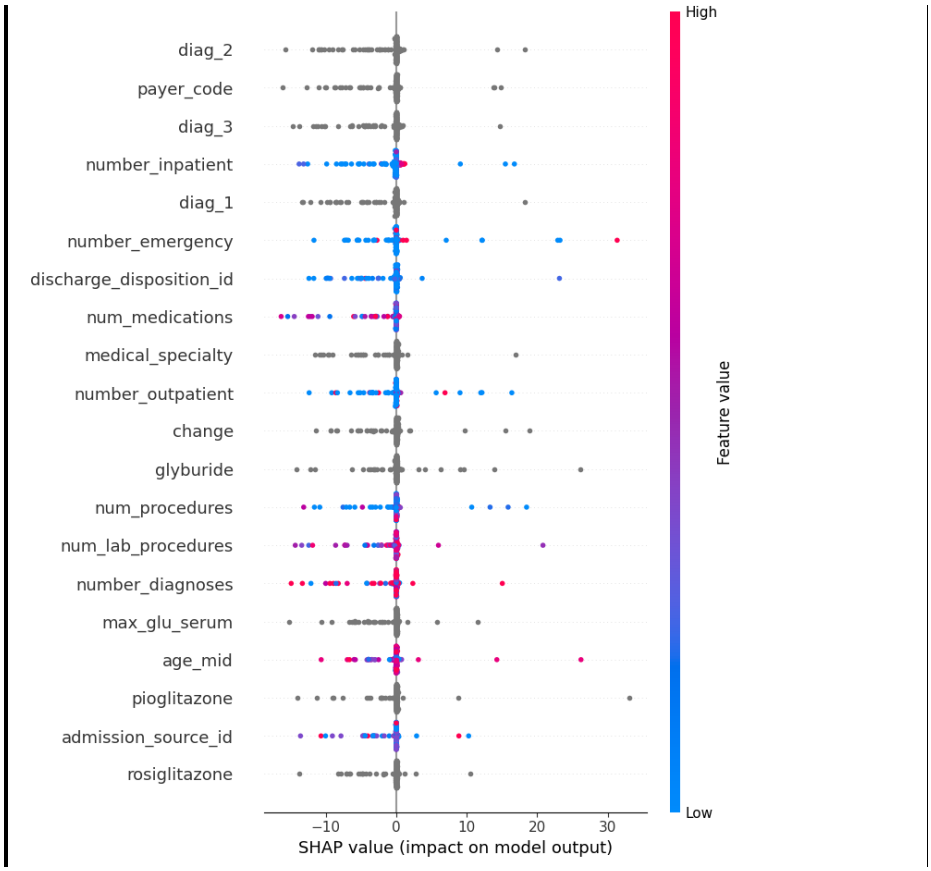
```
MAE: 1.735968360027513
RMSE: 2.3634945348845644
```

[redacted]

[redacted]

[redacted]

[redacted]


diag_2
payer_code
diag_3
number_inpatient
diag_1
number_emergency
discharge_disposition_id
num_medications
medical_specialty
number_outpatient
change
glyburide
num_procedures
num_lab_procedures
number_diagnoses
max_glu_serum
age_mid
pioglitazone
admission_source_id
rosiglitazone
−10
0
10
20
30
SHAP value (impact on model output)
High
Feature value
Low

Prediction probabilities
Not Readmitted 1.00
Readmitted 0.00
Not Readmitted
Readmitted
number_emergency <=...
0.07
discharge_disposition_i...
0.06
number_inpatient <=...
0.05
num_medications <= ...
0.04
admission_type_id <=...
0.03
1.00 < admission_sour...
0.02
number_outpatient <=...
0.02
number_diagnoses <=...
0.00
num_procedures <= 0.00
0.00
31.00 < num_lab_proc...
0.00
Feature Value
number_emergency 0.00
discharge_disposition_id 1.00
number_inpatient 0.00
num_medications 7.00
admission_type_id 1.00
admission_source_id 6.00
number_outpatient 0.00
number_diagnoses 5.00
num_procedures 0.00
num_lab_procedures 37.00

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]